

Order Management System

Serviços em Nuvem · 2026

Disciplina: Serviços em Nuvem
Instituição: Universidade Presbiteriana Mackenzie
Professor: Joaquim Pessoa
Grupo: Alan Ribeiro, Guilherme Shinohara, Kauã Alencar, Kauan Sarzi, Ricardo Kawamuro

1. Introdução

O presente documento descreve a arquitetura, implementação e infraestrutura em nuvem do sistema **Order Management System (OMS)**, desenvolvido como trabalho prático da disciplina de Computação em Nuvem. O sistema implementa operações CRUD completas sobre pedidos (*orders*), com backend em Java Spring Boot, frontend estático, banco de dados PostgreSQL gerenciado pelo Amazon RDS, e uma função serverless AWS Lambda para geração de relatórios estatísticos.

Todos os serviços foram implantados na AWS, dentro de uma VPC privada com sub-redes públicas e privadas, seguindo boas práticas de segurança e isolamento de rede.

2. Arquitetura do Sistema

A arquitetura segue o modelo de três camadas (frontend, backend, banco de dados) integradas por um API Gateway gerenciado, com função Lambda para relatórios.

Diagrama de Arquitetura

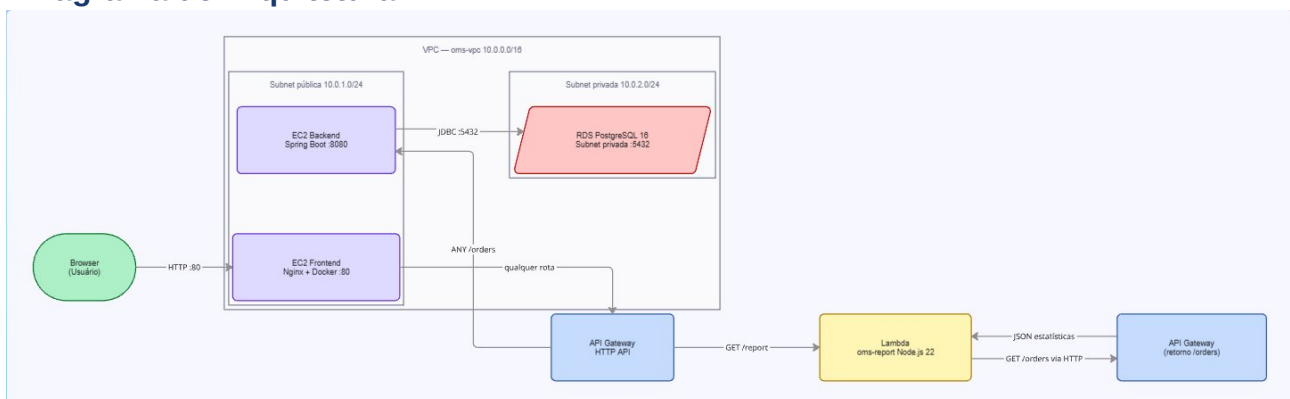


Figura 1 — Diagrama de arquitetura da solução na AWS.

Componentes principais:

Frontend (EC2): Aplicação HTML/CSS/JS servida pelo Nginx em uma instância EC2 na sub-rede pública. Acessa a API via API Gateway.

Backend (EC2 + Docker): API REST desenvolvida em Java Spring Boot, containerizada com Docker. Roda em EC2 na sub-rede pública e acessa o RDS na sub-rede privada.

Amazon RDS (PostgreSQL): Banco de dados PostgreSQL provisionado na sub-rede privada, acessível apenas pelo security group do backend (sg-backend na porta 5432).

API Gateway: HTTP API gerenciado pela AWS com três rotas: ANY /orders, ANY /orders/{proxy+} e GET /report. CORS configurado com stage \$default.

AWS Lambda (Node.js 22.x): Função serverless invocada pela rota GET /report. Consome dados da própria API via HTTP e retorna estatísticas em JSON (total, por status, ticket médio).

VPC + Security Groups: VPC com sub-redes públicas (EC2) e privadas (RDS). Security groups controlam o tráfego: sg-frontend (porta 80), sg-backend (8080), sg-rds (5432 apenas do sg-backend).

3. Evidências no Console AWS

As imagens a seguir evidenciam a configuração dos principais serviços AWS utilizados no projeto.

3.1 Amazon RDS

Instância PostgreSQL criada na sub-rede privada, com Multi-AZ desativado (ambiente de desenvolvimento).

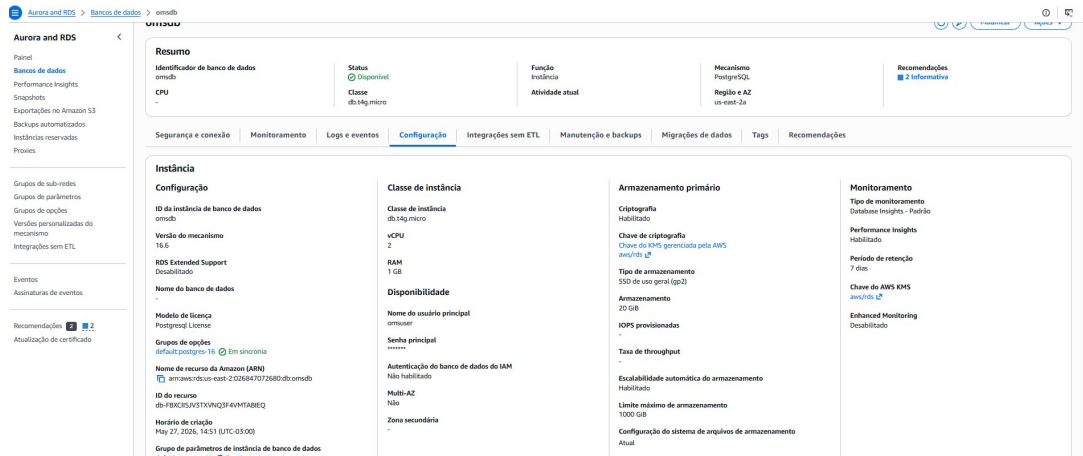


Figura — 3.1 Amazon RDS.

3.2 API Gateway

HTTP API com as rotas configuradas: /orders, /orders/{proxy+} e /report integrada à Lambda.

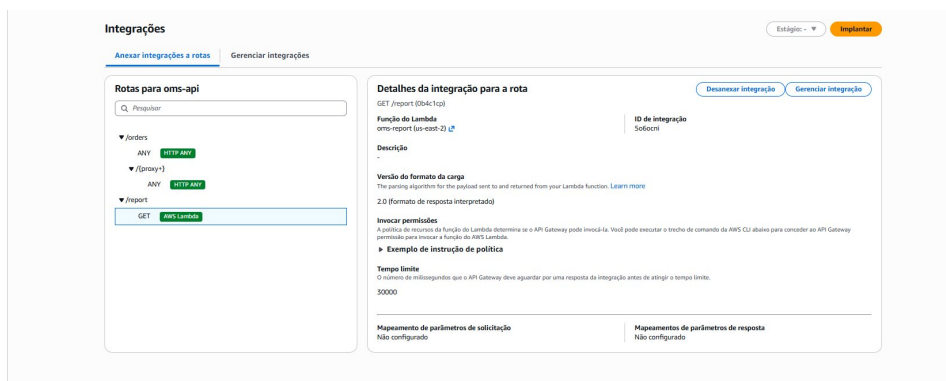


Figura — 3.2 API Gateway.

3.3 AWS Lambda — oms-report

Função Node.js 22.x com variável de ambiente API_URL apontando para o API Gateway.

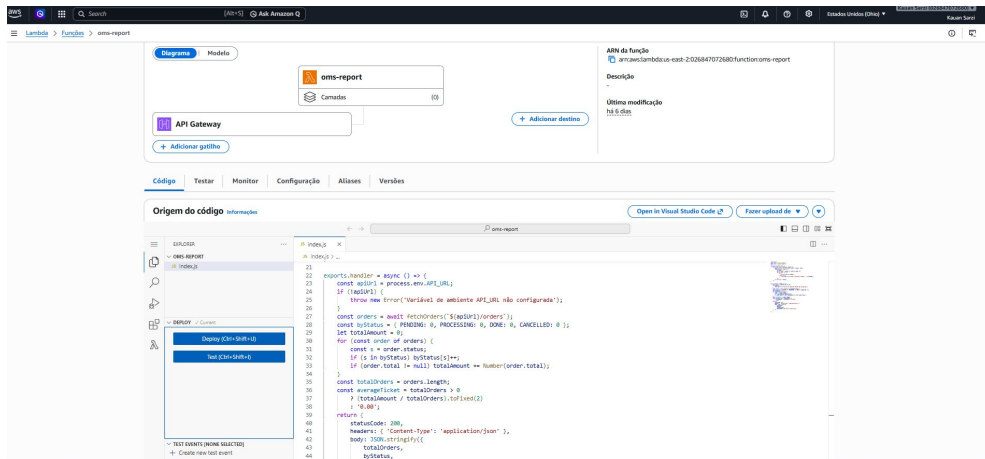


Figura — 3.3 AWS Lambda — oms-report.

3.4 EC2 — Backend

Instância EC2 com container Docker do Spring Boot em execução na porta 8080.

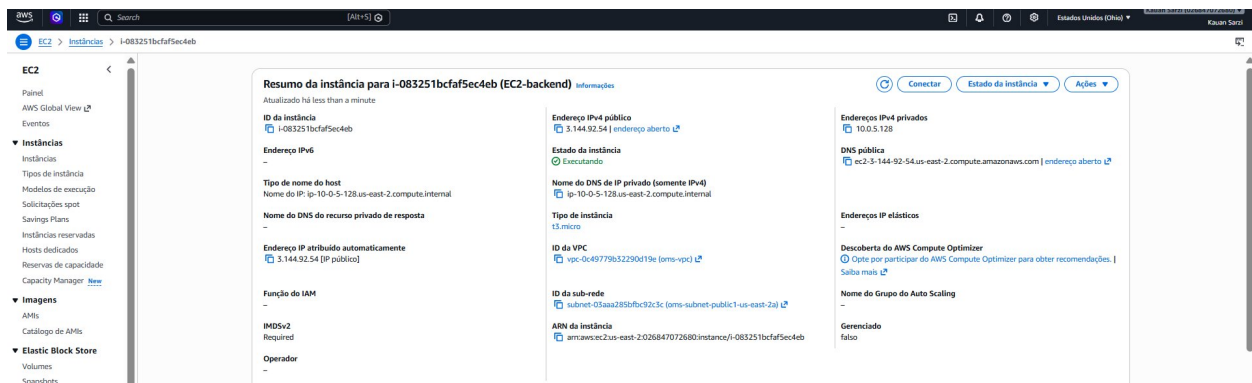


Figura — 3.4 EC2 — Backend.

3.5 EC2 — Frontend

Instância EC2 com container Nginx servindo o frontend na porta 80.

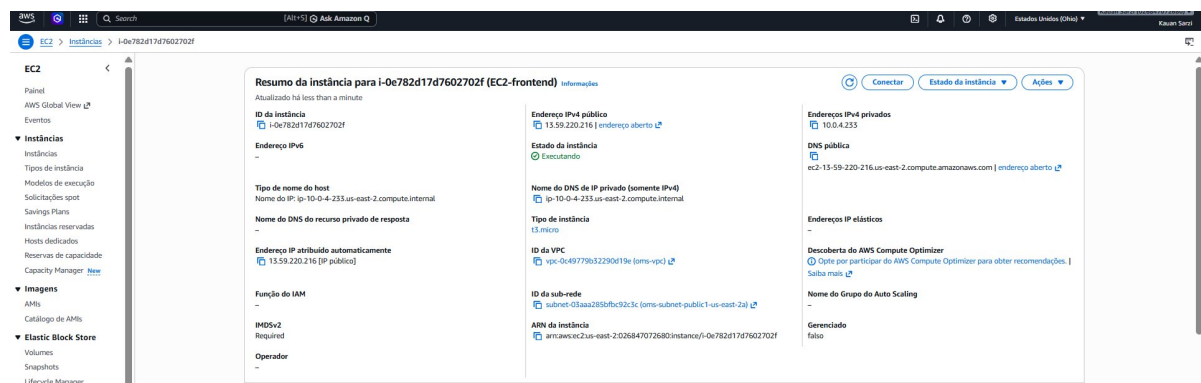


Figura — 3.5 EC2 — Frontend.

4. Detalhes de Implementação

4.1 Backend — Spring Boot

O backend expõe cinco endpoints REST sobre a entidade **Order**:

Método	Rota	Descrição
GET	/orders	Listar todos os pedidos
GET	/orders/{id}	Buscar pedido por ID
POST	/orders	Criar novo pedido
PUT	/orders/{id}	Atualizar pedido existente
DELETE	/orders/{id}	Remover pedido

4.2 Lambda — /report

A função Lambda é invocada pelo API Gateway na rota **GET /report**. Ela consome a rota */orders* do próprio backend via HTTP (usando a variável de ambiente **API_URL**) e retorna um JSON com:

- totalOrders — total de pedidos cadastrados
- byStatus — contagem agrupada por status
- averageTicket — valor médio dos pedidos
- generatedAt — timestamp de geração

4.3 Banco de Dados

O schema SQL define a tabela **orders** com os campos: id (SERIAL PK), customer_name (VARCHAR), product (VARCHAR), quantity (INT), price (NUMERIC), status (VARCHAR) e created_at (TIMESTAMP). O banco é inicializado via script *schema.sql* executado na criação do RDS.

5. Divisão de Tarefas

O trabalho foi dividido entre os 5 integrantes do grupo conforme descrito abaixo:

Membro	Nome	Responsabilidade Principal
Membro 1	Guilherme Shinohara	Backend Spring Boot + testes locais com PostgreSQL
Membro 2	Alan Ribeiro	Dockerfile do backend + deploy na EC2 (backend)
Membros 3 e 4	Kauan Sarzi e Ricardo Kawamuro	VPC, sub-redes, security groups e Amazon RDS
Membros 3 e 4	Ricardo Kawamuro e Kauan Sarzi	API Gateway + AWS Lambda (/report)
Membro 5	Kauã Alencar	Frontend + Dockerfile Nginx + deploy EC2 (frontend)

Nota: Todos os membros participaram das etapas de testes integrados, gravação do vídeo e montagem do ZIP final.

6. Conclusão

O projeto foi implementado com sucesso, atendendo a todos os requisitos da disciplina: CRUD completo com Spring Boot, containerização com Docker, implantação na AWS com VPC privada, banco de dados RDS, API Gateway, função Lambda serverless e frontend estático. A arquitetura adotada segue boas práticas de segurança de rede, com o banco de dados isolado em sub-rede privada e comunicação controlada por security groups.

A experiência permitiu ao grupo compreender na prática os desafios de infraestrutura em nuvem, incluindo configuração de rede, gerenciamento de variáveis de ambiente, integração entre serviços AWS e deploy de aplicações containerizadas.